MERIDIAN HEALTH GROUP | DATA ENGINEERING

Hospital Data Platform

Technical Documentation

Bronze / Silver / Gold analytics pipeline with data-quality controls,
30-day readmission KPI, revenue reconciliation and row-level security.

DOCUMENT VERSION	1.1
DATE	13 October 2026
STATUS	Release
CLASSIFICATION	Internal - synthetic data only
STACK	Python 3.11+, DuckDB, pandas, dbt-duckdb, Airflow, pytest

Document control

Item	Detail
Title	Meridian Hospital Data Platform - Technical Documentation
Version / date	1.1 / 13 October 2026 (documents platform release v1.1)
Subject	A bronze-silver-gold analytics pipeline for a fictional five-branch hospital group (Meridian Health Group)
Audience	Data engineers, analytics engineers, data stewards, platform operators, reviewers and hiring managers
Source of truth	Repository `hospital-data-platform`. Figures were re-taken at release v1.1 (see `CHANGELOG.md`) from a fresh run of the committed demo dataset (seed 42) and a fresh 250k-admission benchmark on the date above.
Data classification	All data is synthetic. No real patients, staff or invoices exist anywhere in the repository.
Related documents	`docs/design_decisions.md`, `docs/runbook.md`, `docs/benchmark.md`, `docs/erd.png`, `docs/architecture.png`, `CHANGELOG.md`

Revision history

Version	Date	Change
1.0	13 Oct 2026	First release of the technical documentation (platform v1.0: 73 tests, 5 mutations).
1.1	13 Oct 2026	Platform v1.1: source-to-bronze reconciliation, tunable alert thresholds (ctl.config), volume and freshness alerts, alert notification, physician-rate confidence intervals, alert failure-path tests (104 tests, 12 mutations), pinned dependencies, lint and CI changes. **Correction:** the access matrix has 13 distinct users, not 12 as v1.0 stated; benchmark re-measured.

How to read this document

- **Sections 1-3** give the purpose, scope and the end-to-end architecture. Start here for a 10-minute overview.
- **Sections 4-8** describe the data: sources, the target model, layer-by-layer processing, business rules and the KPI definitions.
- **Sections 9-13** cover controls: data quality, schema drift, incremental loading, security, monitoring and orchestration.
- **Sections 14-17** cover verification, performance, operation and the honest list of limitations and next steps.
- **Appendices** hold the data dictionary, the exception-rule catalogue, the repository map and a glossary.

Conventions

Convention	Meaning
BR-n	Numbered business rule (Section 7). Each rule is implemented in exactly one SQL file and cited in its header.
SK	Surrogate key. -1 is the Unknown member present in every dimension.
Flag columns	Columns named `flag_*` or `is_*` hold 0/1 exception and eligibility markers. Nothing is silently dropped.
Run	One execution of the pipeline, identified by `run_id` in `ctl.pipeline_run_log`.

Contents

Document control 2

Revision history . . . 2
How to read this document . . . 2
Conventions . . . 2

Contents 3

1. Executive summary 6

2. Business context and objectives 7

2.1 The organisation . . . 7
2.2 Business problems addressed . . . 7
2.3 Objectives and success criteria . . . 7

3. Solution architecture 8

3.1 Logical architecture . . . 8
3.2 Technology stack . . . 9
3.3 Execution flow of a run . . . 9

4. Source systems and ingestion contracts 10

4.1 The change feed and the watermark . . . 10
4.2 Defects deliberately injected by the generator . . . 10

5. Data model 12

5.1 Target star schema . . . 12
5.2 Tables and row counts (demo run) . . . 12
5.3 Modelling principles . . . 12

6. Layer-by-layer processing 14

6.1 Bronze: landing . . . 14
6.2 Schema-drift guard . . . 14
6.3 Source-to-bronze reconciliation . . . 14
6.4 Silver: trusted admissions . . . 15
6.5 Silver/gold: physician attribution (SCD2) . . . 15
6.6 Gold: facts, dimensions and views . . . 15

7. Business rules 16

7.1 Interpretation decisions . . . 16

8. 30-day unplanned readmission KPI 17

8.1 Algorithm . . . 17
8.2 One measure, four grains . . . 17
8.3 Exclusions reconciled . . . 18
8.4 Physician grain: small denominators . . . 18

9. Revenue reconciliation 19

9.1 Silver: trusted billing . . . 19
9.2 Transaction types and sign . . . 19
9.3 The bridge . . . 19
9.4 Orphan invoices . . . 19

10. Data-quality framework 20

10.1 Principles . . . 20
10.2 Exceptions raised in the demo run . . . 20
10.3 Proving the register is complete and precise . . . 20
10.4 Stewardship output . . . 20

11. Row-level security 21

11.1 Design . . . 21
11.2 Scenarios covered by the demo matrix . . . 21

12. Monitoring, logging and operations 22

12.1 Control tables . . . 22
12.2 Monitoring views . . . 22
12.3 Runbook summary . . . 23
12.4 Why 'just re-run it' is safe . . . 23
12.5 Tuning thresholds and getting paged . . . 23

13. Orchestration and dbt 25

13.1 Incremental loading . . . 25
13.2 Airflow DAG . . . 25
13.3 dbt port and equivalence testing . . . 25
13.4 Continuous integration . . . 26

14. Testing and verification strategy 27

14.1 Five independent lines of evidence . . . 27
14.2 Mutation check . . . 27
14.3 Running the tests . . . 28

15. Performance and scalability 29

15.1 Measured benchmark . . . 29

15.2 Scaling beyond the benchmark . . . 29
15.3 Dashboard . . . 29

16. Known limitations, risks and roadmap 31

16.1 Limitations . . . 31
16.2 Risk register . . . 32
16.3 Roadmap . . . 32

17. Installation and operation guide 33

17.1 Prerequisites . . . 33
17.2 Quick start . . . 33
17.3 Querying the warehouse . . . 33
17.4 Makefile targets . . . 33
17.5 Regenerating the synthetic data . . . 33

Appendix A. Data dictionary 34

A.1 gold.fact_admission . . . 34
A.2 gold.fact_readmission_index . . . 34
A.3 gold.fact_billing_transaction . . . 35
A.4 Dimensions . . . 35
A.5 gold.vw_readmission_rate_by_physician . . . 35
A.6 Control tables added in v1.1 . . . 35

Appendix B. Exception-rule catalogue 36

B.2 Alert catalogue . . . 36

Appendix C. Repository map 37

Appendix D. Glossary 37

1. Executive summary

The Meridian Hospital Data Platform turns five messy source extracts from a five-branch hospital group into a trusted admission layer, a 30-day unplanned readmission KPI, a revenue reconciliation and a row-level security model. It is built to be **run, tested and read**: every business rule lives in versioned SQL, every defect the pipeline handles is deliberately planted by a data generator, and every figure in this document comes from a query or a test you can re-run.

Headline results (committed demo dataset, seed 42)

Measure	Result
Admission source rows to distinct to valid	2,681 to 2,571 to 2,528
30-day unplanned readmission rate (group)	6.93% (151 of 2,180 eligible index admissions)
Readmission range by branch	Tabuk 4.73% to Jeddah 8.33%
Billing	2,593 trusted transactions, SAR 24.52m net revenue (invoices 24.66m, credit notes -0.14m, adjustments -0.004m)
Data-quality exceptions raised	306 (34 HIGH, 141 MEDIUM, 131 LOW); every planted defect detected, no false alarms
Source reconciliation	5 of 5 entities MATCH (row counts and money control totals); alerts view empty on a healthy run
Row-level security	19 matrix rows, 13 distinct users (11 appear in the entitlement bridge; 2 are fully deactivated) incl. multi-branch, redundant and revoked grants; 67 expanded entitlement rows
Automated tests	104 tests passing, plus 12 of 12 deliberate mutations detected (business rules, alerts, reconciliation, interval maths, notifier)
Scale	~1.06M rows loaded end to end in about 8.4 s (1 vCPU / 3 GB container; see Section 15 for comparability)

What makes the design notable

- **Grain first.** `gold.fact_admission` has one row per business admission; every mart derives from it, never from bronze.
- **Quarantine, never delete.** Bad rows stay visible for counts and audit but are excluded from the logic they would corrupt.
- **Self-validating.** The generator writes a manifest of every injected defect; tests assert the register catches exactly those keys.
- **Independent oracle.** A pandas re-implementation of the rules, sharing no code with the SQL, is compared row by row with the warehouse.
- **Operable.** Schema-drift gating, watermarks on `AvailableToETL`, source-to-bronze reconciliation, six alert types with thresholds held in a config table, webhook notification, idempotent replay and a runbook.
- **Honest about small numbers.** Physician-level readmission rates carry a 95% Wilson interval and a reportable flag, so a 1-of-1 physician cannot top a league table unchallenged.

Verification status
Everything described as tested was executed for this document: the pipeline ran with 0 failed steps and all 104 tests passed. The Airflow DAG (no Airflow deployment, including its new alert call), the Snowflake / SQL Server reference DDL (`sql/12_production_scale_reference.sql`), and any BI semantic model are **design-reviewed but not run live**. A real chat webhook (tested only against a local HTTP server) and Python 3.11 (the pinned dependency set needs 3.12+) were not verified either. All are flagged again in Section 16.

2. Business context and objectives

2.1 The organisation

Meridian Health Group is a fictional hospital group with five branches - Riyadh, Jeddah, Dammam, Tabuk and Buraidah - each running its own Hospital Information System (HIS). A central billing system, a physician master with assignment history and a security matrix complete the source landscape. The data covers admissions from 1 January 2025 to 22 June 2026.

2.2 Business problems addressed

Problem	Why it matters	Platform answer
There is no single definition of an admission. Source systems resend corrected versions, and test and cancelled encounters leak into volumes.	Every KPI built on a wrong count is wrong.	A deduplicated, flagged silver layer with one row per business admission (BR-1, BR-2).
Readmission is requested as a group KPI, but physicians move between departments and branches.	Attributing outcomes to a physician's *current* assignment misstates performance.	Point-in-time SCD2 attribution (BR-3) and a readmission fact that can be sliced at four grains.
Finance and clinical operations see different monthly revenue.	Both are legitimate (service period vs transaction period) but unexplained differences destroy trust.	One billing fact with two date roles plus a reconciliation bridge that sums exactly (BR-8).
Branch directors must see only their data; people change roles and leave.	Over-exposure of patient-level data is a compliance risk.	A matrix-driven entitlement bridge honouring the latest snapshot and ignoring inactive grants.
Source systems change without notice and late corrections arrive after the fact.	Silent drops or opaque failures at 2am.	Schema-drift gating, an `AvailableToETL` watermark, idempotent replay and monitoring views.

2.3 Objectives and success criteria

#	Objective	Measured by
O1	Trusted admission grain	Uniqueness of `AdmissionID`; no admission silently dropped; valid set equals the oracle
O2	Correct 30-day unplanned readmission KPI	Every index row equals the oracle; 30-day boundary correct to the minute
O3	Explainable revenue	Bridge components sum exactly to service-period revenue every month
O4	Enforceable access control	Per-user visibility equals the oracle for all matrix users
O5	Unattended operation	Idempotent re-runs; N incremental batches equal one full load; replay lands 0 rows
O6	Provable data-quality coverage	Recall = 100% and precision = 100% against the planted-defect manifest

3. Solution architecture

3.1 Logical architecture

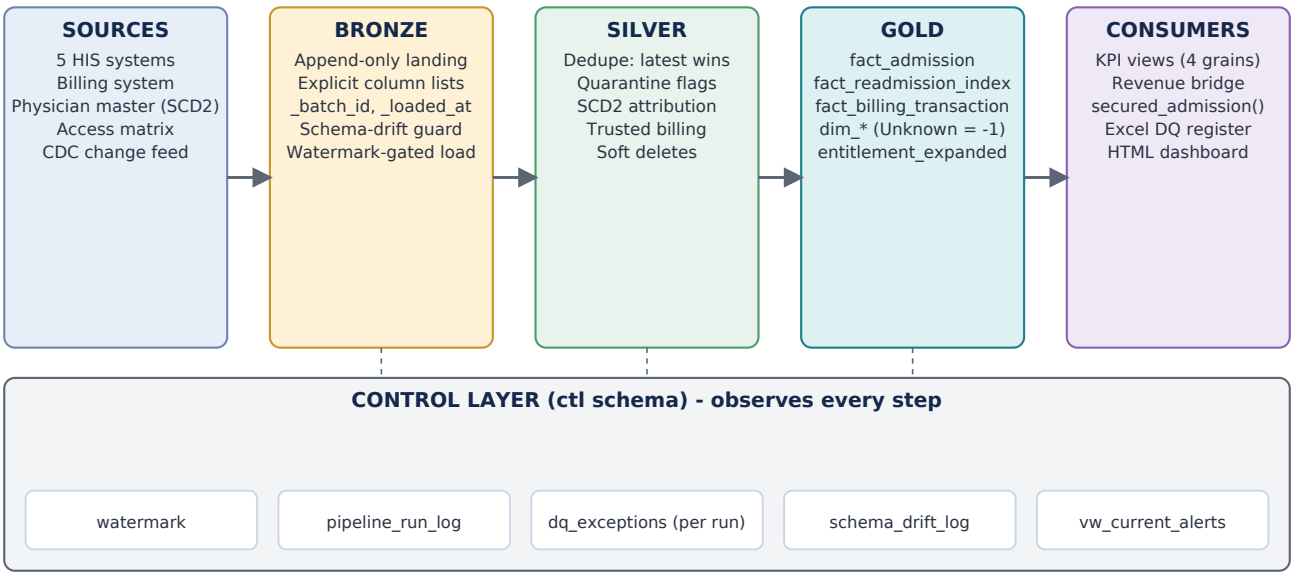


Figure 1 - End-to-end architecture: sources, bronze, silver, gold, consumers and the control layer.

The platform follows a medallion pattern. Data flows one way - bronze to silver to gold - and a separate control layer observes every step. All business logic is SQL; Python only orchestrates (ordering, logging, failure handling, export).

Layer	Schema	Purpose	Key properties
Bronze	`bronze`	Land the 5 source extracts as received	Append-only; explicit column lists; loader metadata `_batch_id`, `_loaded_at`; no business logic
Silver	`silver`	Trusted, deduplicated, flagged records	Latest-version wins; validity and quarantine flags; SCD2 attribution
Gold	`gold`	Star schema, KPI views, RLS bridge	Surrogate keys, Unknown members, single KPI definition re-sliced at every grain
Control	`ctl`	Operate and observe the pipeline	Watermark, run log, DQ register (history per run), schema snapshot and drift log, source-to-bronze reconciliation, tunable thresholds (ctl.config), monitoring views

3.2 Technology stack

Component	Technology	Role and rationale
SQL engine	DuckDB (>= 1.1)	In-process analytical engine: the entire platform is executable and testable in seconds, with no infrastructure.
Orchestration of steps	Python 3.11+ (`run_pipeline.py`)	Runs SQL files in order, logs each step to `ctl.pipeline_run_log`, fails loudly, never swallows errors.
Transformation port	dbt-duckdb (>= 1.8)	Maintainable production form of silver and gold: 13 models, 32 tests and four equivalence tests.
Scheduler	Apache Airflow 2.9+ / 3.x (TaskFlow API)	Hourly DAG with retries, single active run and a health gate.
Test framework	pytest + independent pandas oracle	104 tests; oracle comparison, defect recall/precision, edge cases, operational properties, alert failure paths.
Reporting	Static HTML dashboard, Excel DQ register, matplotlib figures	Self-contained outputs generated from the gold layer.
Alerting	Webhook (Slack-compatible JSON)	`notify.py` posts current alerts as one message; silent when healthy; delivery failure never fails the pipeline.
CI	GitHub Actions	Lint job; pinned Python 3.12 leg (pipeline, pytest, incremental demo, dbt build, mutation check); unpinned Python 3.11 compatibility leg.
Quality tooling	ruff, pre-commit, `constraints.txt`	Lint and import order; exact dependency pins verified to reproduce the environment (Python 3.12+).

3.3 Execution flow of a run

```
 1  Schema guard      compare each incoming file with ctl.schema_snapshot  (INFO / BLOCKING)
 2  Bronze load       append rows past the watermark for entities that passed the guard
 3  Reconciliation    compare bronze with the source FILE: row counts + money control totals
 4  03  silver        deduplicate admissions, flag orphan / negative LOS / missing discharge / deletes
 5  04  silver/gold   physician SCD2 dimension and point-in-time attribution
 6  05  gold          dim_date, dim_branch, dim_department, dim_patient
 7  06  gold          fact_admission
 8  07  gold          fact_readmission_index and the four KPI views (physician view with CI)
 9  08  gold          billing_trusted, fact_billing_transaction, revenue views and bridge
10  09  gold          entitlement_expanded and secured_admission() macro
11  10  control       append DQ exceptions for this run_id
12  11  control       monitoring views (run health, DQ trend, drift, volume, freshness, alerts)
13  Export            marts and control tables to CSV; Excel DQ register; dashboard
14  (optional) notify post ctl.vw_current_alerts to $ALERT_WEBHOOK_URL when non-empty
```

The watermark advances only after bronze *and* silver/gold have succeeded (`advance_watermark()` runs at the end of the `transform` task), so a failed run can be retried without skipping data.

4. Source systems and ingestion contracts

Five extracts represent the source landscape. The generator produces them deterministically from a seed (same seed and size give byte-identical files).

Extract	Bronze table	Grain	Business key	Version column	Demo rows
`Admissions.csv`	`bronze.fact_admissions`	One row per admission *source version*	AdmissionID	SourceVersion, LastModifiedDateTime	2,681
`Physician_Assignments.csv`	`bronze.dim_doctors_scd`	One row per physician assignment window (SCD2)	DoctorNatural Key + EffectiveFrom	SourceLastUpdated	85
`Billing_Extract.csv`	`bronze.billing_transactions`	Invoice, credit note or adjustment line	TransactionID	LastModifiedDateTime	2,629
`Access_Matrix.csv`	`bronze.security_matrix_rls`	User x access grant; full snapshot per batch	UserEmail + Branch + Department	Latest `_batch_id`	19
`Change_Feed.csv`	`bronze.source_change_log`	CDC-style event: when a change became available to ETL	ChangeSequence	AvailableToETL	5,403

4.1 The change feed and the watermark

The change feed records, for every source row, `SourceLastModified` (when the source changed) and `AvailableToETL` (when the ETL could first see it). Incremental loading is keyed on the latter. Using `SourceLastModified` would silently skip a record modified at time T that propagated to the ETL later - a late-arriving correction. The watermark per (source, entity) advances to the maximum `AvailableToETL` actually processed, never to `now()`.

```
CREATE OR REPLACE VIEW ctl.vw_next_batch AS
SELECT c.*
FROM bronze.source_change_log c
LEFT JOIN ctl.watermark w ON w.source_system = c.SourceSystem AND w.entity = c.Entity
WHERE c.AvailableToETL > COALESCE(w.last_watermark_ts, TIMESTAMP '1900-01-01');
```

4.2 Defects deliberately injected by the generator

Real hospital extracts are messy, so the generator is adversarial on purpose. Each defect is injected at a configurable rate and recorded in `data/ground_truth.json` with the exact business keys affected.

Defect	Treatment in the platform
Duplicate source versions, including same-version ties	Latest wins (`SourceVersion` DESC, then `LastModifiedDateTime` DESC); history stays in bronze
Test and cancelled encounters	Excluded from valid admissions; retained in silver for traceability
Source-side deletes arriving via the change feed	Soft delete (`SOURCE_DELETED`), effective only once the event has arrived
Orphan physician keys	Mapped to synthetic Unknown member `DoctorSK = -1`; HIGH exception
Negative or missing length of stay	Kept for volume counts; quarantined from date-sequenced (readmission) logic
Overlapping and gapped physician SCD2 windows	Latest correction wins (HIGH); nearest prior assignment for gaps (MEDIUM)
Admission after a recorded death	Both rows retained; flagged for MRN reconciliation (HIGH)

Defect	Treatment in the platform
Late-arriving corrections	Watermark on `AvailableToETL`, not on modified time
Corrected invoices, credit notes, adjustments, late and orphan invoices	Latest version wins; sign preserved; bridged; orphans flagged but counted
Schema drift (added, removed, retyped columns)	Added = INFO; removed or unsafely retyped = BLOCKING for that entity only

Operational failures are exercised by the tests rather than planted in the data: a truncated extract with valid columns, a step that raises, a column removed from a file, a HIGH-severity rate above threshold, a silent source, and money altered in transit with the row count unchanged. Section 12 describes the alert for each.

5. Data model

5.1 Target star schema

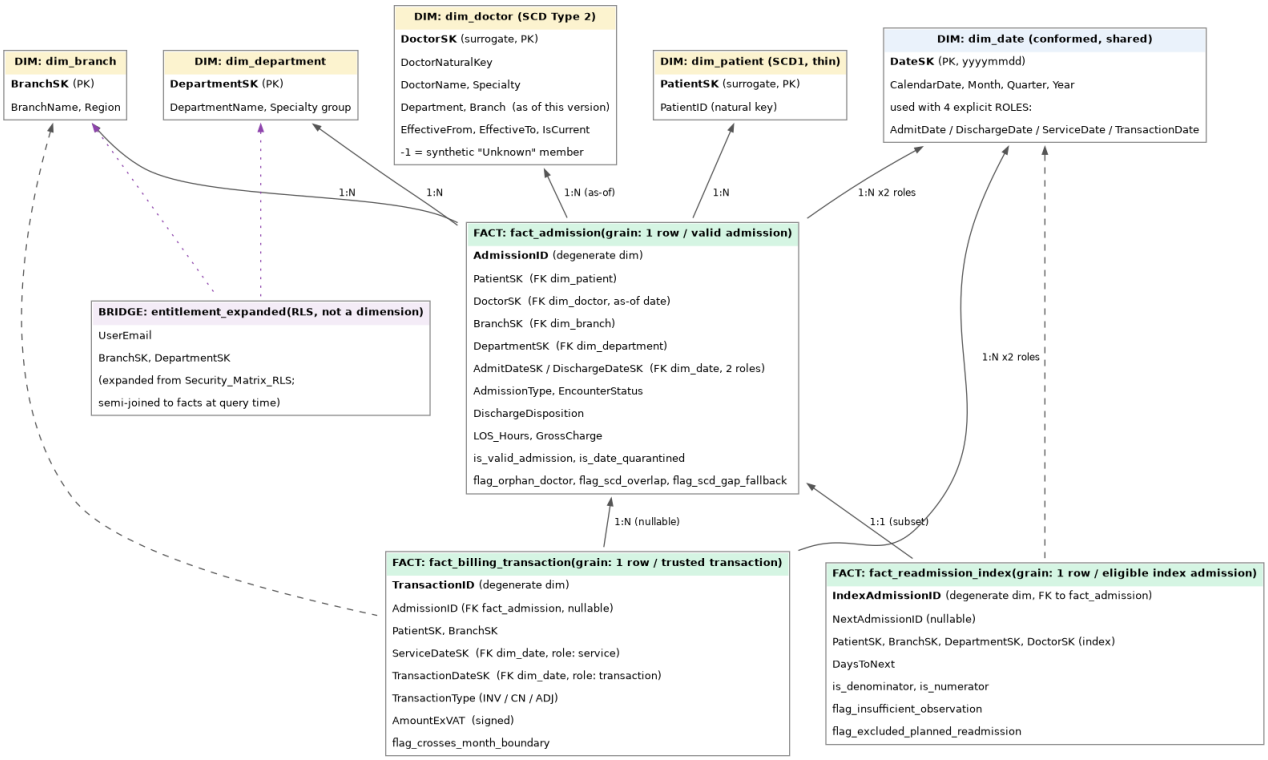


Figure 2 - Gold-layer entity-relationship diagram.

5.2 Tables and row counts (demo run)

Table	Grain	Rows	Notes
`gold.fact_admission`	One row per business admission	2,571	Carries all exception flags; every mart derives from it
`gold.fact_readmission_index`	One row per eligible index admission	2,276	2,180 in denominator + 96 with insufficient observation
`gold.fact_billing_transaction`	One row per latest transaction version	2,593	Two date roles: service date and transaction date
`gold.dim_doctor`	One row per physician assignment window (SCD2)	86	85 SCD rows + Unknown member
`gold.dim_patient`	One row per patient	2,126	Includes patients seen only in billing; Unknown member
`gold.dim_branch`	One row per branch	6	5 branches + Unknown; region mapping (Central, Western, Eastern, Northern)
`gold.dim_department`	One row per department	7	6 departments + Unknown; specialty group (Medical, Surgical, Emergency)
`gold.dim_date`	One row per calendar day	1,095	Whole years covering the data; Friday/Saturday weekend (KSA)
`gold.entitlement_expanded`	User x branch x department	67	Pre-expanded RLS bridge

5.3 Modelling principles

- **Grain first.** The grain of each fact is stated in its file header and asserted in tests (`test_fact_admission_grain_is_unique`).

- **Unknown members.** Every dimension has `SK = -1`; facts use `COALESCE(sk, -1)`, so a failed lookup never loses a row.
- **One billing fact, two date roles.** Forking into two facts would duplicate every transaction through shared dimensions and start a 'which one is right' debate.
- **Physician as SCD2.** The physician dimension is keyed by assignment window; facts carry the SK in effect on the event date.
- **Deterministic surrogate keys.** Assigned with `ROW_NUMBER()` at build time because the gold layer is rebuilt as a whole; production uses identity keys maintained by `MERGE` (see Section 16).

6. Layer-by-layer processing

6.1 Bronze: landing

Bronze tables mirror the source columns plus `_batch_id` and `_loaded_at`. A correction lands as a new row, so silver and gold can be rebuilt from bronze without re-pulling the source. Loads use **explicit column lists**; a positional insert would silently misalign data if a source added a column mid-file (found and fixed during design review).

6.2 Schema-drift guard

Before every bronze load, `schema_guard.py` compares the incoming file's column names and coarse type families with `ctl.schema_snapshot` (the last-known-good schema).

Outcome	Severity	Behaviour
Identical schema	-	Load proceeds
Column added	INFO	Load proceeds. The new column is *not* loaded (explicit column lists) until the DDL is changed deliberately, so it cannot silently alter a business number
Column removed	BLOCKING	That entity's load is skipped and logged; other entities still load
Type changed	BLOCKING unless safe widening	Safe transitions: INTEGER to BIGINT/DOUBLE, DATE to TIMESTAMP, and identity. Everything else blocks

The snapshot is refreshed only when a load proceeds, so a blocked file is compared against the same trusted baseline on the next run.

6.3 Source-to-bronze reconciliation

The run log shows that each step moved *some* rows; it cannot show that it moved *all* of them. After each bronze load the pipeline compares what landed with the source file itself and records one row per entity in `ctl.source_reconciliation`:

Status	Meaning
MATCH	Row count agrees and, where the entity has a money column, the control total agrees (`GrossCharge` for admissions, `AmountExVAT` for billing)
MISMATCH	Rows were dropped, duplicated or altered on the way in. Raises `SOURCE_RECONCILIATION_MISMATCH`
PENDING	Incremental run and the change feed still has rows past this batch's cutoff, so equality is not yet expected
BLOCKED	The entity was halted by breaking schema drift. Counted by rows only (its money column may be the column that disappeared); the drift alert already covers it, so it is not also reported as a mismatch

What 'landed' means. For a full load it is the rows tagged with this run's batch id (a re-run appends to bronze by design, so the cumulative pile is not compared). For an incremental load it is the cumulative bronze rows. The access matrix is a full snapshot per batch, so its latest snapshot is compared. The money control total matters because a mangled amount with an unchanged row count passes every count check; this is tested explicitly.

Cost. The check re-reads each source file; on the 1.06M-row benchmark it took about 0.6 to 0.9 s.

> **Bug found while building this**
> The first version summed the money column of every file. An entity blocked because that very column had been removed made the reconciliation crash. The pre-existing schema-guard test caught it; blocked entities are now counted by rows only.

6.4 Silver: trusted admissions

`silver.admission_trusted` has exactly one row per `AdmissionID`. It is produced in three steps:

- **Deduplicate** with `ROW_NUMBER() OVER (PARTITION BY AdmissionID ORDER BY SourceVersion DESC, LastModifiedDateTime DESC)`. Versions are never summed or averaged.
- **Flag** orphan physician keys, completed encounters with no discharge, negative length of stay and source-side deletes. Flags are columns, not filters.
- **Derive** `is_valid_admission` (not Cancelled/Test and not deleted) and `is_date_quarantined` (missing discharge or negative LOS).

```
CASE WHEN EncounterStatus IN ('Cancelled', 'Test') OR flag_source_deleted = 1
     THEN 0 ELSE 1 END AS is_valid_admission,
CASE WHEN flag_missing_discharge = 1 OR flag_negative_los = 1
     THEN 1 ELSE 0 END AS is_date_quarantined
```

A source delete counts only when its change event's `SourceLastModified` is at or after the admission's latest version and the event is present in the change log, so a delete cannot take effect before it has arrived.

6.5 Silver/gold: physician attribution (SCD2)

Business rule BR-3 attributes each admission to the physician assignment *effective on the admission date*. `EffectiveTo` is an **inclusive calendar date**, so the join tests `AdmissionDateTime < EffectiveTo + 1 day`.

Case	Detection	Resolution	Severity
Exact match	One assignment covers the date	Use it	-
Overlap	Two or more assignments cover the date (back-dated correction)	Row with the latest `SourceLastUpdated`	HIGH
Gap	No assignment covers the date	Nearest *prior* assignment (never the current record)	MEDIUM
Unresolved	No prior assignment at all	`DoctorSK = -1`	-
Orphan key	Key absent from the SCD entirely	`DoctorSK = -1`	HIGH

> **Design-review finding**
> An earlier version treated `EffectiveTo` as an exclusive midnight timestamp, pushing every admission on the last day of an assignment into a false coverage gap. It is now inclusive, with a regression test, and the mutation check re-introduces this exact bug to prove the suite catches it.

6.6 Gold: facts, dimensions and views

`gold.fact_admission` joins silver admissions to the attribution result and to the conformed dimensions, producing the business admission fact with all flags. `gold.fact_readmission_index` and the revenue structures are described in Sections 8 and 9.

7. Business rules

Rules are numbered, implemented once and cited in the SQL file headers, so a reviewer can trace a number on a report to the exact predicate.

ID	Rule	Implemented in
BR-1	A valid admission is the latest source version of an encounter, excluding Cancelled, Test and source-deleted encounters.	`03`
BR-2	Latest version wins: highest `SourceVersion`, tie-break latest `LastModifiedDateTime`. Versions are never summed or averaged.	`03`, `08`
BR-3	Historical activity is attributed to the physician assignment effective on the event date, never the current record.	`04`
BR-4	An eligible readmission index is valid, non-quarantined, and its disposition is not Died or Transfer.	`07`
BR-5	Planned (Elective) admissions are excluded as the *candidate readmission*; the first *unplanned* admission within 30 days is matched.	`07`
BR-6	The readmission search is not branch-scoped (a Jeddah discharge followed by a Riyadh admission counts).	`07`
BR-7	Index discharges within the trailing 30 days of the data horizon lack a full window: flagged and excluded from the denominator.	`07`
BR-8	Credit notes and adjustments keep their sign and belong to the month they were raised.	`08`

7.1 Interpretation decisions

Two requirements are ambiguous in the wild and the platform commits to an explicit interpretation, isolated in a single predicate so the alternative is a one-line change:

- **Where does the planned-admission exclusion apply?** To the *candidate readmission* (CMS-style). An elective return is not an unplanned bounce-back, but the index admission itself may be elective.
- **Is a transfer a readmission?** No. A transfer continues the same episode of care, so Transfer and Died dispositions cannot be index admissions.

8. 30-day unplanned readmission KPI

8.1 Algorithm

```
eligible_index = valid AND not date-quarantined AND disposition NOT IN ('Died','Transfer')

in_window      = eligible_index idx  JOIN  fact_admission nxt
                 ON same PatientID AND nxt.AdmissionID <> idx.AdmissionID
                 AND nxt valid AND not date-quarantined
                 AND nxt.AdmissionDateTime >  idx.DischargeDateTime
                 AND nxt.AdmissionDateTime <= idx.DischargeDateTime + 30 days

unplanned      = first in_window row where next type <> 'Elective'   (ROW_NUMBER by admit time)
planned        = first in_window row where next type  = 'Elective'

is_denominator = idx.DischargeDateTime <= as_of - 30 days
is_numerator   = is_denominator AND unplanned exists
as_of          = MAX(LastModifiedDateTime) in gold.fact_admission
```

The index fact is auditable end to end: each row shows the index admission, the matched readmission (if any), days to next, the first elective admission in the window and the flags `flag_insufficient_observation` and `flag_excluded_planned_readmission`.

8.2 One measure, four grains

A single pair of measures, `SUM(is_denominator)` and `SUM(is_numerator)`, is re-sliced at group, branch, department and physician level through four views. A test enforces that all four grains reconcile to the same totals, so the KPI cannot be defined two different ways in two reports.

Branch	Eligible index admissions	Unplanned readmissions	Rate
Riyadh	689	44	6.39%
Jeddah	564	47	8.33%
Dammam	325	25	7.69%
Buraidah	306	21	6.86%
Tabuk	296	14	4.73%
Group	**2,180**	**151**	**6.93%**

Department	Eligible index admissions	Unplanned readmissions	Rate
Cardiology	330	37	11.21%
Emergency	547	38	6.95%
Internal Medicine	429	25	5.83%
Neurology	121	7	5.79%
Orthopedics	234	14	5.98%
Surgery	519	30	5.78%

Department rates are computed from the department view totals (sum across branches). Index-department attribution means a readmission is credited to the department of the index stay.

8.3 Exclusions reconciled

Population	Count	Handling
Index rows in `fact_readmission_index`	2,276	Valid, non-quarantined, not Died/Transfer
of which in the denominator	2,180	Discharged at least 30 days before the as-of date
of which insufficient observation	96	Excluded from the denominator; re-evaluated automatically when later data lands
Index rows whose only return was elective	56	`flag_excluded_planned_readmission = 1`; not counted as readmissions

8.4 Physician grain: small denominators

The same measures are available per physician, where small denominators make a raw rate misleading: one readmission out of one stay is '100%'. `gold.vw_readmission_rate_by_physician` therefore keeps **every** physician (nothing is hidden, and the rows still sum to the group total) but adds a 95% **Wilson score interval** and an `is_reportable` flag, and lists reportable physicians first.

- **Interval.** Wilson rather than the textbook normal approximation, which collapses to a zero-width interval at 0% and 100% - exactly where small samples live. Reference values guarded by tests: 0 of 10 = 0.00% to 27.75%; 1 of 10 = 1.79% to 40.42%; 5 of 10 = 23.66% to 76.34%.
- **Reportable.** Eligible index admissions at or above `min_physician_volume` (default 25, held in `ctl.config`) *and* a resolved physician. The synthetic UNKNOWN physician is never reportable.
- **One rule, one place.** The dashboard previously hard-coded its own cut-off of 25; it now filters on the flag.

Physician (demo data)	Eligible	Readmitted	Rate	95% interval	Reportable
D060	35	7	20.00%	10.04% to 35.89%	Yes
D004	31	6	19.35%	9.19% to 36.28%	Yes
D026	27	4	14.81%	5.92% to 32.48%	Yes
D029	16	3	18.75%	6.59% to 43.01%	No (below 25)
UNKNOWN (unresolved key)	1	1	100.00%	20.65% to 100.00%	Never

Across the demo data the view has 88 physician / branch / department rows: 48 reportable (26 to 53 eligible admissions) and 40 not (1 to 24). Note that D029's rate is close to D060's, but its interval is about ten points wider.

What this does not do
There is no case-mix adjustment: `dim_patient` holds no age, sex or diagnosis, so branch and department differences (for example Tabuk 4.73% vs Jeddah 8.33%) may partly reflect different patient populations. Treat the rates as monitoring signals, not league tables.

9. Revenue reconciliation

Finance typically reports revenue by **service period** (when care was delivered); operations often see revenue by **transaction period** (when it was billed or adjusted). Both are correct. The platform stores one fact and exposes both aggregations plus a bridge that explains the difference.

9.1 Silver: trusted billing

Invoices are corrected after landing, so `silver.billing_trusted` keeps the version with the latest `LastModifiedDateTime` per `TransactionID` (BR-2). 2,629 source rows reduce to 2,593 transactions.

9.2 Transaction types and sign

Type	Meaning	Count	Net amount (SAR)	Period rule
INV	Invoice	2,477	24,660,312.18	Service date and transaction date may differ
CN	Credit note / reversal	68	-136,977.75	Sign preserved; belongs to the month raised (BR-8)
ADJ	Adjustment	48	-4,129.24	Sign preserved; belongs to the month raised (BR-8)
Total		**2,593**	**24,519,205.19**	

9.3 The bridge

`gold.vw_revenue_reconciliation_bridge` partitions each service month's revenue into four components that always sum to total service-period revenue:

- **recognized_same_month** - invoices billed in the month of service;
- **invoiced_in_later_month** - invoices billed after the service month (pure timing difference);
- **credit_notes_and_reversals** - credit notes (negative);
- **adjustments** - adjustments (signed).

Service month	Same month	Invoiced later	Credit notes	Adjustments	Total
2025-01	581,364.06	688,734.64	-3,685.15	0.00	1,266,413.55
2025-02	456,889.47	811,828.33	-4,604.59	-1,474.89	1,262,638.32
2025-03	678,410.18	803,211.80	-694.43	692.73	1,481,620.28

First three service months shown (SAR, ex-VAT).

9.4 Orphan invoices

Five transactions reference an `AdmissionID` absent from the admissions extract. They stay in revenue (the cash is real) but carry `flag_orphan_admission = 1` and a MEDIUM `ORPHAN_BILLING_ADMISSION` exception routed to billing / HIS reconciliation.

> **Why not two facts?**
> A 'service-period fact' and a 'transaction-period fact' would each carry every transaction. Joined through a shared patient or branch dimension they double-count, and every disagreement between them becomes an unexplained argument. One fact with two date keys (`ServiceDateSK`, `TransactionDateSK`) plus a bridge is both smaller and self-explaining.

10. Data-quality framework

Data quality is a first-class output, not a side effect. Every exception is (a) a flag on the affected row and (b) a row in `ctl.dq_exceptions`. The register is generated *from the same flags the pipeline acted on*, so it cannot drift out of sync with what actually happened.

10.1 Principles

- **Quarantine, never delete.** Impossible rows stay in the fact for volume counts but are excluded from date-sequenced logic.
- **Append-only by run.** The register keeps one set of rows per `run_id`, enabling trend analysis (an earlier full rebuild gave a trend with a single point).
- **Every rule has a treatment.** The register records what the pipeline did, not just what it found.
- **Severity drives routing.** HIGH goes to master-data stewards; MEDIUM to source teams; LOW is informational.

10.2 Exceptions raised in the demo run

Severity	Rule code	Count	Entity
HIGH	ORPHAN_DOCTOR_FK	10	Admissions
HIGH	MISSING_DISCHARGE	8	Admissions
HIGH	NEGATIVE_LOS	8	Admissions
HIGH	SCD_OVERLAP	4	Physician_Assignments
HIGH	ILLOGICAL_ADMIT_AFTER_DEATH	4	Admissions
MEDIUM	DUP_SOURCE_VERSION	124	Admissions and Billing
MEDIUM	SOURCE_DELETED	8	Admissions
MEDIUM	ORPHAN_BILLING_ADMISSION	5	Billing_Extract
MEDIUM	SCD_COVERAGE_GAP	4	Physician_Assignments
LOW	INSUFFICIENT_OBSERVATION_WINDOW	96	Fact_Readmission_Index
LOW	NON_INPATIENT_ENCOUNTER	35	Admissions
Total		**306**	34 HIGH, 141 MEDIUM, 131 LOW

10.3 Proving the register is complete and precise

The generator writes `ground_truth.json` listing every planted defect key per rule. Tests assert, per rule, that the set of keys in the register equals the planted set - **recall** (nothing missed) and **precision** (nothing extra). The manifest also lists SCD cases that *must not* be flagged (20 in the demo set), guarding against false positives such as the inclusive end-date regression.

10.4 Stewardship output

`pipeline/export_dq_register.py` produces a formatted Excel workbook (`docs/data_quality_exception_register.xlsx`) for stewards, and the dashboard summarises exceptions caught.

11. Row-level security

Access is driven by `Access_Matrix.csv`. A user's visibility is the **union of all active grants**:

Access level	Visibility
Enterprise	All branches and departments
Branch	All departments of the named branch
Department	The named branch + department pair; a user may hold several across different branches

11.1 Design

- **Inactive grants never grant access.** `IsActive = 1` is filtered at the base view, not left for callers to remember.
- **Latest snapshot wins.** The matrix is a full snapshot per batch; only the highest `_batch_id` is used, so revocations take effect on the next load.
- **Expanded bridge.** Grants are expanded once into `gold.entitlement_expanded` (user x branch x department) and facts are semi-joined to it, instead of evaluating a correlated predicate per row. The same pattern maps to a Power BI RLS bridge, a SQL Server SECURITY POLICY over an inline TVF, or a Snowflake row-access policy backed by a mapping table.
- **Redundant grants cannot duplicate rows.** The bridge is DISTINCT and the access path uses EXISTS.

```
-- Secured access path: what a report or semantic layer runs for the signed-in user
SELECT count(*) FROM gold.secured_admission('riyadh.director@mhg.example');
```

11.2 Scenarios covered by the demo matrix

User	Scenario	Expanded rows
enterprise.admin / quality.director	Enterprise access	19 each
riyadh.director, jeddah.director	Branch access (Jeddah also holds a redundant Department grant)	5 each
dammam, tabuk, buraidah directors	Branch access	3 each
cardio.head	Same department across three branches	3
ops.multi	Departments across different branches	3
im.head	Single department grant (Riyadh, Internal Medicine)	1
mixed.access	Active Buraidah grant plus a revoked Riyadh grant (revoked ignored)	3
former.employee, former.contractor	All grants inactive	0 (absent from bridge)

The matrix has 19 rows for 13 distinct users; 11 appear in the bridge (67 rows). The test suite compares per-user visibility with the oracle for every user in the matrix, and a mutation that honours revoked grants is detected.

> **Scope note**
> RLS is modelled at the data layer. Production healthcare deployments additionally need column-level masking for patient identifiers, encryption, access audit logging and retention controls; these are out of scope for this synthetic-data project (see Section 16).

12. Monitoring, logging and operations

12.1 Control tables

Table	Content
`ctl.watermark`	Per (source, entity): `last_watermark_ts` (max AvailableToETL processed), `last_run_id`, `updated_at`
`ctl.pipeline_run_log`	One row per step per run: layer, table, status, rows in/out/rejected, start/end, error message
`ctl.dq_exceptions`	One row per (run, entity, business key, rule): severity, description, treatment
`ctl.schema_snapshot`	Last-known-good column names and type families per entity
`ctl.schema_drift_log`	Every detected drift: type, column, severity, run
`ctl.source_reconciliation`	One row per entity per run: source rows, bronze rows, money control totals, status (MATCH / MISMATCH / PENDING / BLOCKED)
`ctl.config`	Tunable thresholds as data: `min_physician_volume` (25), `high_dq_rate_threshold_pct` (5), `volume_drop_alert_pct` (30), `freshness_max_lag_hours` (0 = off). Seeded once, never overwritten by a re-run; read by the views on every query

12.2 Monitoring views

View	Answers
`ctl.vw_run_health`	Per run: duration, steps total / succeeded / failed, last error
`ctl.vw_dq_trend`	Exception counts by severity per run (the operating SLA for data quality)
`ctl.vw_schema_drift_trend`	Drift events per run, entity and severity
`ctl.vw_volume_drift`	Each extract's row count versus the previous run, as a percentage change
`ctl.vw_source_freshness`	Per source system: latest AvailableToETL and hours since
`ctl.vw_current_alerts`	Single 'is anything wrong right now?' check. **Empty result = healthy**

`vw_current_alerts` raises six alert types. Thresholds come from `ctl.config`, so tuning one is an UPDATE, not a deploy:

- **PIPELINE_FAILED** - any step in the latest run has status FAILED;
- **BLOCKING_SCHEMA_DRIFT** - an entity was halted by a breaking schema change;
- **HIGH_SEVERITY_DQ_RATE** - HIGH exceptions exceed a percentage of valid admissions (default 5%). *Rate-based*, so it scales with volume; the default is illustrative and should be tuned from run history;
- **SOURCE_RECONCILIATION_MISMATCH** - bronze does not match the source file (rows or money control total);
- **SOURCE_VOLUME_DROP** - an extract shrank by more than a percentage versus the previous run (default 30%). A half-empty file with the right columns passes the schema guard; this catches it;
- **STALE_SOURCE** - a source system has produced nothing for more than N hours. Off by default (0) because the demo data is historical and would alert forever.

12.3 Runbook summary

Failure point	Meaning	Action
Schema guard returns BLOCKING	Source column removed or retyped unsafely	Do not force the load. Inspect the drift log; fix at source or update DDL and silver column lists, then re-baseline the snapshot once the change is confirmed intentional. Other entities already loaded.
Bronze insert fails	Corrupt or partial extract	Re-run. Bronze is append-only and downstream is 'latest version wins', so a clean re-pull never double-counts; the watermark advances only at the end.
Silver or gold step fails	Transformation bug or a new edge case	Query bronze for the offending rows, encode the correct treatment as an explicit rule (never patch data in place), re-run. The fix applies retroactively to all history.
Dashboard looks stale but watermark moved	Downstream refresh problem	Pipeline is fine; escalate to the BI refresh schedule.
ROW_NOT_IN_CHANGE_LOG warning	A source row has no change-feed entry and will never be picked up incrementally	Raise with the source team; run a full load in the meantime.
SOURCE_RECONCILIATION_MISMATCH	Bronze differs from the source file (rows dropped, duplicated or amounts altered)	Query ctl.source_reconciliation for the entity and run; re-pull the extract and re-run. If it persists, diff the file against bronze by business key. Do not publish.
SOURCE_VOLUME_DROP	An extract is much smaller than last run but structurally valid	Ask the source team whether it is complete before trusting any KPI. If the drop is genuine (a branch closed), raise the threshold or accept the alert once.
STALE_SOURCE	A source has produced no changes for longer than the threshold	Check the source's extract job and the change feed.
HIGH_SEVERITY_DQ_RATE	HIGH exceptions exceed the configured share of valid admissions	Group the latest run's HIGH exceptions by rule; usually one rule dominates (a new orphan-key pattern, a bad extract).

Breaking schema drift is deliberately **not** retried: a partial source outage should degrade gracefully rather than take the whole batch down.

12.4 Why 'just re-run it' is safe

Silver and gold are `CREATE OR REPLACE`, rebuilt deterministically from append-only bronze, and every dedupe rule is 'latest version wins'. Nothing accumulates downstream, so:

- re-running after a **transient** failure reproduces the state a successful first attempt would have produced - no duplicates, no half-updated gold;
- re-running after fixing a **logic** bug applies the corrected rule to all history without a separate backfill job.

This is asserted by `test_pipeline_is_idempotent`, `test_incremental_batches_converge_to_the_full_load` and `test_replaying_the_last_batch_lands_nothing`.

12.5 Tuning thresholds and getting paged

Thresholds are data. To change one:

```
SELECT * FROM ctl.config;
UPDATE ctl.config SET value = '40' WHERE param = 'volume_drop_alert_pct';
```

Caveat: `ctl.config` has no audit column yet, so record the change and its reason elsewhere (open item in Section 16).

`run_pipeline.py --notify` and the Airflow quality gate post `ctl.vw_current_alerts` to `$ALERT_WEBHOOK_URL` as one Slack-compatible message. Three deliberate behaviours:

- **Silence when healthy.** A daily 'all good' message trains people to ignore the channel.
- **No webhook configured:** the alerts are printed and the function reports why nothing was sent, so local runs and CI stay side-effect free.
- **Delivery failure never raises.** A chat outage must not turn a healthy run into a failed one, nor hide the real failure behind a notification error; it is logged and returned.

Tested against a stub sender, a real local HTTP server and an unreachable URL. **Not tested** against a real chat tool or paging service, and there is no de-duplication, so a persistent condition re-alerts on every run.

13. Orchestration and dbt

13.1 Incremental loading

`run_pipeline.py --mode incremental --batches N` simulates production CDC. Each batch loads only the rows whose `AvailableToETL` is past the per-(source, entity) watermark and up to a cutoff; the watermark then advances to the maximum value actually processed. Silver and gold are rebuilt from the accumulated bronze, so **N incremental batches converge to exactly the same gold state as one full load**.

```
python pipeline/run_pipeline.py                          # full load
python pipeline/run_pipeline.py --mode incremental --batches 4
python pipeline/demo_incremental.py                      # CDC == full load; replay lands 0 rows
```

13.2 Airflow DAG

Task	Responsibility
`land_bronze`	Schema-drift check and watermark-gated incremental load into bronze
`transform`	Silver, gold, DQ register and monitoring via the SQL layer; advances the watermark only after success
`data_quality_gate`	Sends the alerts to the webhook (if configured), then fails the run if `ctl.vw_current_alerts` is non-empty
`dbt_build`	Runs the dbt port with its 32 tests including equivalence tests (only when the gate passes)
`publish`	Exports marts, the Excel DQ register and the dashboard

Settings: hourly schedule, `catchup=False`, `max_active_runs=1`, 2 retries with a 5-minute delay, 30-minute execution timeout. Written for the Airflow 2.9+ / 3.x TaskFlow API with an import fallback between the two.

> **Verification status**
> The DAG is validated structurally by `tests/test_dag.py` and its task logic is exercised by the suite, but it has **not** been run on a live Airflow deployment. Treat it as a reviewed design; the new `notify_alerts` call in the quality gate is likewise untested on a live scheduler. Note also that `land_bronze` loads everything past the watermark rather than a bounded data interval, which is appropriate for the historical demo data but should be changed for interval-based backfills.

13.3 dbt port and equivalence testing

The silver and gold layers are re-implemented as **13 dbt models** (`dbt-duckdb`, tables materialised into `dbt_silver` / `dbt_gold`) with **32 schema tests** (uniqueness, not-null, relationships, accepted values). The DQ register and monitoring views remain in SQL.

Because the same logic now exists twice, four singular tests assert that the dbt output equals the plain-SQL pipeline's tables row for row (EXCEPT in both directions) for `fact_admission`, `fact_readmission_index`, `fact_billing_transaction` and `dim_doctor`. The SQL layer is the executable specification; dbt is the maintainable production form.

13.4 Continuous integration

```
name: ci            # on: push, pull_request
  lint:   pip install -r requirements-dev.txt ; ruff check .
  test (matrix):
    Python 3.12  pinned    pip install -r requirements.txt -c constraints.txt
    Python 3.11  unpinned  pip install -r requirements.txt      # pins need Python >= 3.12
    both:   python pipeline/run_pipeline.py
            python -m pytest tests -q
            python pipeline/demo_incremental.py
            cd dbt && dbt build --profiles-dir .
    3.12 only: python scripts/mutation_check.py
```

14. Testing and verification strategy

The suite contains **104 tests**. Its design goal is that a wrong business rule fails a test, not just a crashing query.

Test module	Tests	Focus
`test_pipeline.py`	41	Grain and integrity; warehouse-vs-oracle row-by-row; planted-defect recall and precision; KPI grain reconciliation; RLS per user; idempotency
`test_incremental.py`	8	CDC batches equal full load; replay is a no-op; deletes honoured only after arrival; watermark behaviour
`test_generator.py`	7	Determinism and manifest consistency
`test_edge_cases.py`	7	30-day boundary to the minute; elective-then-emergency return; transfer index; cancelled return; assignment boundary day; observation window
`test_schema_guard.py`	5	Additive, removed and retyped columns; baseline preserved after a block
`test_dag.py`	4	Structural checks on the Airflow DAG
`test_dbt.py`	1	Runs `dbt build` (13 models, 32 tests incl. equivalence); skipped if dbt-duckdb is absent
`test_enhancements.py`	31	Physician interval vs an independent reference; reconciliation (clean, dropped rows, altered amount, re-run, incremental PENDING to MATCH); **a failure-path test for every alert type** plus a tripwire for untested alerts; notification (stub, real local HTTP server, failures); dependency pins

14.1 Five independent lines of evidence

Line of evidence	What it proves	Limitation
Independent oracle (`tests/oracle.py`, pandas, no shared code)	The SQL implements the stated rules: valid set, latest versions, SCD attribution, every readmission index row, monthly revenue, RLS per user	Shares the author's reading of the requirements; a misread rule would be mirrored in both
Planted-defect manifest	The register catches exactly the injected defects (recall and precision)	Only defects the author thought to plant
Hand-built edge cases	Boundary behaviour that random data rarely hits	Finite hand-selected cases
Operational properties	Idempotency, CDC convergence, replay, delete timing, drift handling	Exercised on DuckDB only
Alert failure-path tests	Each alert fires when, and only when, its condition holds; thresholds are tunable; a tripwire fails if an alert is added without a test	Faults are injected, not real incidents; delivery tested against a local server only

14.2 Mutation check

`python scripts/mutation_check.py` copies the repository, breaks one business rule at a time and requires the suite to fail. All 12 mutations are killed (the original five business-rule mutations plus seven for the operational features):

#	Mutation	Target file
1	EffectiveTo treated as exclusive (false coverage gaps)	`04_doctor_scd_attribution.sql`
2	Elective returns counted as unplanned readmissions	`07_gold_readmission_kpi.sql`
3	Oldest source version wins instead of latest	`03_silver_admission_trusted.sql`
4	Revoked (IsActive = 0) grants still honoured	`09_security_rls.sql`
5	Credit notes lose their sign in the bridge	`08_gold_revenue_reconciliation.sql`
6	Volume-drop alert fires on growth instead of shrinkage	`11_monitoring.sql`

#	Mutation	Target file
7	DQ-rate alert ignores ctl.config (hard-coded 5%)	`11_monitoring.sql`
8	Reconciliation ignores the money control total	`run_pipeline.py`
9	Wilson upper bound computed with the wrong sign	`07_gold_readmission_kpi.sql`
10	Unresolved (UNKNOWN) physician becomes reportable	`07_gold_readmission_kpi.sql`
11	A failed webhook delivery crashes the pipeline	`notify.py`
12	Healthy runs still send a message	`notify.py`

A surviving mutation would mean a rule that no test protects, so this check measures the *sensitivity* of the suite rather than just its pass rate.

14.3 Running the tests

```
pip install -r requirements.txt -c constraints.txt
python pipeline/run_pipeline.py
python -m pytest tests -q            # 104 passed (about 100 s)
python scripts/mutation_check.py     # 12/12 mutations killed
```

15. Performance and scalability

15.1 Measured benchmark

Re-measured for v1.1 on a 1-vCPU / 3 GB container (Python 3.12.3, DuckDB 1.5.6, pandas 3.0.2), without tuning. Reproduce with `make bench`; results vary by machine.

Stage	Wall time
`generate_data.py --admissions 250000` (single-threaded Python)	~63 s
`run_pipeline.py --no-export` (bronze, silver, gold, DQ register, monitoring)	~8.4 s (two runs: 8.42 s and 8.48 s)

Comparability. The v1.0 document reported about 52 s and about 6 s, measured on a different machine (2 vCPU / 7 GB, Python 3.13). The two sets of figures are therefore **not comparable** and no slowdown should be inferred. What can be said for v1.1: source-to-bronze reconciliation accounts for about 0.6 to 0.9 s of the pipeline step (it re-reads each source file), and the pipeline reports 0 failed steps, all five entities MATCH and an empty alerts view at 1.06M rows.

Benchmark dataset / result	Value
Admission source rows / distinct / valid	265,872 / 254,934 / 250,684
Billing rows / change-feed events / SCD rows	261,246 / 528,609 / 741
Total rows loaded	~1.06 million
Group readmission rate (eligible index admissions)	7.77% (218,325)
DQ exceptions raised	27,823

15.2 Scaling beyond the benchmark

The logic and idempotency guarantees do not change at larger volumes; cost and blast radius do. Silver and gold are full rebuilds per batch, which is fine to tens of millions of rows. At 500M+ rows the reference design in `sql/12_production_scale_reference.sql` applies:

Concern	SQL Server target	Snowflake target
Physical design	Monthly partition function and scheme on admission date; clustered columnstore for facts	Clustering keys on admission date; micro-partition pruning
Incremental loads	`MERGE` keyed on business key, driven by watermark	Streams and tasks over bronze; `MERGE` into gold
Surrogate keys	`IDENTITY` maintained by MERGE	Sequences / identity columns
Idempotency	From MERGE keys rather than full rebuild	From MERGE keys and stream offsets

Verification status
The production reference DDL has not been executed against SQL Server or Snowflake. Some DuckDB-flavoured constructs (`DATEDIFF('day', ...)`, `SELECT * EXCLUDE`, `ORDER BY ALL`, `CREATE OR REPLACE TABLE ... AS`) need dialect translation, so porting is not free.

15.3 Dashboard



Figure 3 - Self-contained dashboard generated from the gold layer (docs/dashboard.html).

16. Known limitations, risks and roadmap

16.1 Limitations

#	Limitation	Impact	Mitigation / next step
L1	Silver and gold are full rebuilds per batch; surrogate keys are assigned at build time	Keys may change between runs; cost grows with volume	MERGE with identity or hash keys; add a test that keys survive re-runs
L2	Oracle and pipeline share one author's interpretation	A misread requirement passes both	Clinical sign-off on BR-4 to BR-7; compare with an external reference implementation
L3	Engine is DuckDB only; production DDL never executed	Portability is claimed, not proven	Run on Snowflake trial or SQL Server in Docker; add a CI job
L4	Airflow DAG never run live; `land_bronze` ignores `data_interval_end`	Backfill semantics untested	Run in Docker Compose or Astro in CI; bound loads by interval
L5	As-of date = `MAX(LastModifiedDateTime)`	A late correction can move the observation horizon	Use the extract timestamp from the source contract
L6	Schema guard compares name and coarse type family; an entirely blank column can look retyped	Possible false BLOCKING	Replace with a data contract or schema registry
L7	Alert thresholds are tunable but the defaults are illustrative; webhook delivery was tested only against a local HTTP server; no de-duplication	A mis-set threshold, or a page that never arrives or repeats every run	Tune from run history; test against the real chat or paging tool; add de-duplication
L8	RLS modelled only at the data layer	No masking, encryption or audit logging	Column-level masking, access audit table, retention policy
L9	Static HTML dashboard; no BI semantic model	Limited self-service	Power BI / Looker semantic layer using the entitlement bridge
L10	`ctl.config` changes are not audited	A threshold can be loosened without a record	Add changed_by / changed_at and a history table
L11	Python 3.11 was not run here; the pinned dependency set requires Python 3.12+	The 3.11 CI leg is unverified and runs unpinned	Confirm the 3.11 leg in CI; decide whether to keep supporting 3.11
L12	No case-mix adjustment: `dim_patient` has no age, sex or diagnosis	Branch and department rates may reflect patient mix	Add risk-adjustment attributes before comparing providers

16.2 Risk register

Risk	Likelihood	Impact	Control in place
Source schema changes unannounced	High	High	Schema guard with per-entity blocking; INFO for additive changes
Late-arriving corrections skipped	Medium	High	Watermark on AvailableToETL; idempotent rebuild
Wrong physician attribution after reassignment	Medium	Medium	SCD2 point-in-time join; overlap and gap flags; regression tests
Over-exposure of data through stale access grants	Medium	High	Latest-snapshot-wins; inactive grants filtered at base; per-user oracle tests
Rule regression during refactor	Medium	High	Oracle comparison, mutation check, dbt equivalence tests
Silent data loss	Low	High	No-admission-dropped test; quarantine instead of delete; run log rows in/out
Truncated or partial extract with valid columns	Medium	High	Source-to-bronze reconciliation (counts and money totals); volume-drop alert; per-entity run log
Small-sample physician rates misread as performance	Medium	Medium	Wilson 95% interval and is_reportable flag (minimum volume in ctl.config); totals still reconcile
Dependency drift changes results	Medium	Medium	Exact pins in constraints.txt (Python 3.12 leg); mutation check in CI; unpinned 3.11 leg flags incompatibilities

16.3 Roadmap

Phase	Item	Outcome
1	Execute the pipeline on a real warehouse (Snowflake or SQL Server) in CI	Portability proven
1	Incremental gold with MERGE and stable surrogate keys	Cost proportional to change, not volume
2	Run the DAG in a containerised Airflow in CI; interval-bounded loads	Orchestration verified
2	Data contracts in place of the heuristic schema guard	Fewer false blocks; explicit ownership
2	Route alerts to the on-call tool with de-duplication; verify against a real webhook; audit ctl.config changes	Operational response, not just detection
3	PHI controls: masking policies, access audit, retention	Production-grade security posture
3	BI semantic model over the gold layer using the entitlement bridge	Self-service analytics

17. Installation and operation guide

17.1 Prerequisites

- Python 3.11 or newer (3.12 or newer to use the exact pinned versions in `constraints.txt`)
- Packages from `requirements.txt`: duckdb >= 1.1, pandas >= 2.0, numpy >= 1.26, openpyxl >= 3.1, matplotlib >= 3.8, pytest >= 8.0, dbt-duckdb >= 1.8 (optional)
- Graphviz (`dot`) only if you want to regenerate the architecture image

17.2 Quick start

```
pip install -r requirements.txt -c constraints.txt     # exact, verified versions (Python 3.12+)
python pipeline/run_pipeline.py              # full load -> output/warehouse.duckdb + CSV marts
python pipeline/run_pipeline.py --notify     # same, and post any alerts to $ALERT_WEBHOOK_URL
python -m pytest tests -q                    # 104 tests
python pipeline/demo_incremental.py          # CDC batches == full load; replay lands 0 rows
python pipeline/export_dq_register.py        # formatted Excel issue register
python pipeline/build_dashboard.py           # docs/dashboard.html (self-contained)
cd dbt && dbt build --profiles-dir .         # dbt port incl. equivalence tests (run the pipeline first)
python pipeline/generate_data.py --admissions 250000   # scale up (seeded, deterministic)
```

17.3 Querying the warehouse

```
duckdb output/warehouse.duckdb

SELECT * FROM gold.vw_readmission_rate_by_branch;
SELECT * FROM gold.vw_revenue_reconciliation_bridge ORDER BY service_month;
SELECT * FROM ctl.vw_current_alerts;                      -- empty = healthy
SELECT * FROM ctl.source_reconciliation ORDER BY run_id DESC, entity;
SELECT * FROM gold.vw_readmission_rate_by_physician WHERE is_reportable = 1;
SELECT count(*) FROM gold.secured_admission('riyadh.director@mhg.example');
```

17.4 Makefile targets

Target	Action
`make run`	Full load to `output/`
`make test`	Run the 104-test suite
`make lint`, `dev`	Run ruff; install the (pinned) linter
`make lock`	Regenerate `constraints.txt` from a clean virtual environment (review the diff, then run the suite)
`make incremental`	CDC demonstration
`make dq`, `figures`, `dashboard`	Excel register, summary figure, dashboard
`make dbt`	Build the dbt port (requires `make run` first)
`make mutation`	Rule mutation check
`make bench`	250k-admission scale benchmark
`make all`	lint, run, test, dq, figures, dashboard, incremental, dbt
`make clean`	Remove `output/` and pytest cache

17.5 Regenerating the synthetic data

`generate_data.py` accepts `--admissions`, `--seed`, `--start`, `--end` and `--out`. The same seed and size always give byte-identical files. The committed demo set uses seed 42 and about 2,500 requested admissions; regenerating it with `make data` overwrites the committed extracts and manifest, so commit them together.

Appendix A. Data dictionary

A.1 gold.fact_admission

Column	Type	Description
AdmissionID	VARCHAR	Business key; unique (grain)
PatientID / PatientSK	VARCHAR / INT	Source patient id and surrogate key
SourceSystem	VARCHAR	Originating HIS (six values incl. a test source)
BranchSK, DepartmentSK, DoctorSK	INT	Conformed dimension keys; -1 = Unknown. DoctorSK is point-in-time
AdmitDateSK, DischargeDateSK	INT	yyyymmdd keys into dim_date
AdmissionDateTime, DischargeDateTime	TIMESTAMP	Encounter timestamps (discharge may be NULL)
AdmissionType	VARCHAR	Emergency, Urgent, Elective
EncounterStatus	VARCHAR	Completed, Cancelled, Test
DischargeDisposition	VARCHAR	e.g. Home, Transfer, Died
los_hours	BIGINT	Length of stay in hours; NULL when impossible or missing
GrossCharge	DECIMAL(14,2)	Gross charge from the source
SourceVersion, LastModifiedDateTime	INT / TIMESTAMP	Winning source version
is_valid_admission	0/1	BR-1 outcome
is_date_quarantined	0/1	Missing discharge or negative LOS
flag_orphan_doctor, flag_missing_discharge, flag_negative_los, flag_source_deleted	0/1	Silver exception flags
flag_scd_overlap, flag_scd_gap_fallback	0/1	Attribution exception flags

A.2 gold.fact_readmission_index

Column	Description
index_admission_id, PatientID, PatientSK	Index admission and patient
BranchSK, DepartmentSK, index_branch, index_department, index_doctor_sk	Index attribution used for all KPI slicing
index_admission_type, index_discharge_dt	Index type and discharge time
next_admission_id, next_admission_dt, next_admission_type, next_branch, days_to_next	First *unplanned* admission within 30 days (NULL if none)
planned_admission_id_in_window	First elective admission in the window, if any
as_of_ts	Observation horizon used for this row
flag_insufficient_observation	Discharged within 30 days of the as-of date
is_denominator, is_numerator	KPI measure pair, summed at every grain
flag_excluded_planned_readmission	Only an elective return occurred in the window

A.3 gold.fact_billing_transaction

Column	Description
TransactionID, AdmissionID, PatientID, Branch, TransactionType	Keys and type (INV, CN, ADJ)
PatientSK, BranchSK	Dimension keys
ServiceDate, TransactionDate, ServiceDateSK, TransactionDateSK	The two date roles
AmountExVAT	Signed amount excluding VAT
InvoiceReference, RelatedTransactionID	Invoice id; link to the transaction a CN/ADJ relates to
flag_crosses_month_boundary	Service month differs from transaction month
flag_linked_to_prior_transaction	RelatedTransactionID is present
flag_orphan_admission	AdmissionID absent from admissions

A.4 Dimensions

Table	Key columns	Attributes
dim_doctor	DoctorSK (PK); DoctorNaturalKey	DoctorName, Specialty, Department, Branch, EffectiveFrom, EffectiveTo (inclusive), IsCurrent, SourceLastUpdated
dim_patient	PatientSK; PatientID	FirstSeenDate
dim_branch	BranchSK; BranchName	Region (Central, Western, Eastern, Northern)
dim_department	DepartmentSK; DepartmentName	SpecialtyGroup (Medical, Surgical, Emergency)
dim_date	DateSK (yyyymmdd); CalendarDate	Year, Quarter, Month, MonthName, MonthStart, DayOfWeek (0 = Sunday), IsWeekend (Fri/Sat)

A.5 gold.vw_readmission_rate_by_physician

Column	Description
DoctorNaturalKey, DoctorName, Branch, Department	Physician and the index branch / department the admissions are attributed to
eligible_index_admissions, unplanned_readmissions	Sums of the denominator and numerator flags
readmission_rate_pct	100 x numerator / denominator
ci95_lower_pct, ci95_upper_pct	95% Wilson score interval, in percent, clamped to 0 to 100
is_reportable	1 when eligible admissions >= min_physician_volume and the physician is resolved (not UNKNOWN)

A.6 Control tables added in v1.1

Table / column	Description
ctl.config (param, value, description)	Tunable thresholds; value is text and cast where used; read through the macro ctl.cfg(param)
ctl.source_reconciliation.run_id, entity, mode	Run, short entity name, full or incremental
source_rows, bronze_rows	Rows in the source file; rows landed (this load for full, cumulative for incremental)
source_amount, bronze_amount	Control total of the money column (NULL for entities without one, and for blocked entities)
status, checked_at	MATCH / MISMATCH / PENDING / BLOCKED; time of the check (a re-check appends, the latest wins)

Appendix B. Exception-rule catalogue

Rule code	Sev.	Entity	Condition	Treatment
DUP_SOURCE_VERSION	MED	Admissions, Billing	More than one version landed for the key	Latest version retained; earlier versions kept in bronze
NON_INPATIENT_ENCOUNTER	LOW	Admissions	EncounterStatus is Cancelled or Test	Excluded from is_valid_admission; retained in silver
ORPHAN_DOCTOR_FK	HIGH	Admissions	DoctorNaturalKey not in physician assignments	DoctorSK = -1; admission retained; master-data investigation
NEGATIVE_LOS	HIGH	Admissions	Discharge earlier than admission	Kept for counts; quarantined from readmission logic
MISSING_DISCHARGE	HIGH	Admissions	Completed encounter with NULL discharge	Kept for counts; quarantined from readmission logic
SOURCE_DELETED	MED	Admissions	DELETE event arrived after the latest version	Soft delete; excluded from valid; retained for audit
SCD_OVERLAP	HIGH	Physician_Assignments	Two assignments cover the admission date	Latest SourceLastUpdated wins; flagged (BR-3)
SCD_COVERAGE_GAP	MED	Physician_Assignments	No assignment covers the admission date	Nearest prior assignment; flagged for stewards
ORPHAN_BILLING_ADMISSION	MED	Billing_Extract	AdmissionID not in admissions	Kept in revenue; flagged; routed to reconciliation
ILLOGICAL_ADMIT_AFTER_DEATH	HIGH	Admissions	Valid admission after a prior valid encounter with disposition Died	Both retained; MRN / identity reconciliation
INSUFFICIENT_OBSERVATION_WINDOW	LOW	Fact_Readmission_Index	Index discharge within 30 days of as-of	Excluded from denominator; re-evaluated when data lands

B.2 Alert catalogue

Alert	Condition	Default	Config parameter
PIPELINE_FAILED	A step of the latest run has status FAILED	-	-
BLOCKING_SCHEMA_DRIFT	Breaking drift halted an entity in the latest run	-	-
HIGH_SEVERITY_DQ_RATE	HIGH exceptions / valid admissions above the threshold	5%	high_dq_rate_threshold_pct
SOURCE_RECONCILIATION_MISMATCH	Bronze differs from the source file in the latest run	-	-
SOURCE_VOLUME_DROP	Extract shrank by more than the threshold vs the previous run	30%	volume_drop_alert_pct
STALE_SOURCE	Hours since a source's latest change above the threshold	0 (off)	freshness_max_lag_hours

Appendix C. Repository map

```
hospital-data-platform/
  data/        synthetic extracts (5 CSV) + ground_truth.json           seed 42, ~2.5k admissions
  sql/         00 drift control | 01 bronze DDL | 02 control tables + ctl.config + reconciliation
               03 silver admissions | 04 SCD attribution | 05 dimensions | 06 fact_admission
               07 readmission KPI | 08 revenue reconciliation | 09 RLS
               10 DQ register | 11 monitoring and alerts | 12 production reference (not executed)
  pipeline/    generate_data, run_pipeline, schema_guard, notify, demo_incremental,
               export_dq_register, build_dashboard, make_figures
  dbt/         dbt-duckdb port: 4 silver + 9 gold models, schema tests, 4 equivalence tests
  airflow/     dags/meridian_pipeline.py        hourly incremental DAG
  tests/       oracle.py + 104 tests (pipeline, incremental, edge cases, generator, drift, DAG, dbt, enhancements)
  scripts/     mutation_check.py
  docs/        architecture, ERD, design decisions, runbook, benchmark, dashboard, DQ register
  .github/     workflows/ci.yml (lint; pinned 3.12 leg; 3.11 compatibility leg)
  root         constraints.txt | requirements-dev.txt | pyproject.toml | .pre-commit-config.yaml | CHANGELOG.md
  Makefile     lint | lock | dev | run | test | incremental | dq | figures | dashboard | dbt | mutation | bench | all
```

Appendix D. Glossary

Term	Definition
Bronze / Silver / Gold	Medallion layers: raw landing, trusted and flagged, business-ready star schema
CDC	Change data capture; here a change feed with an `AvailableToETL` timestamp
Watermark	High-water mark of data already processed per (source, entity)
SCD2	Slowly changing dimension type 2: history kept as validity windows
Index admission	The admission from which a 30-day readmission window is measured
Unplanned readmission	A non-Elective admission within 30 days after an index discharge
Quarantine	Retain a row for counts and audit but exclude it from logic it would corrupt
Unknown member	Synthetic dimension row with key -1 so that facts never lose rows to failed lookups
Oracle	An independent implementation of the rules used to check the warehouse
Mutation check	Deliberately breaking a rule to prove the test suite notices
RLS	Row-level security: users see only rows they are entitled to
HIS	Hospital Information System, the per-branch source of admissions
MRN	Medical record number, the patient identity key within a hospital
SAR	Saudi riyal, the currency of the billing amounts
Control total	A sum of a money column compared between the source file and bronze, to catch altered values that a row count cannot
Reconciliation	Checking that what was loaded equals what the source sent
Wilson score interval	A confidence interval for a proportion that stays sensible at small samples and at 0% or 100%
Compatibility leg	A CI job that runs the oldest supported Python without pinned dependencies